

1. bitAnd: 德摩根定律。
2. bitXor: 分别取同为 1 和同为 0 的位, 再合并, 再取反
3. sameSign: 需要处理普通正负数和 0。先提取 x 和 y 的符号位, 判断符号位是否不同, 不同直接返回 0。处理 0 的情况时判断是否至少有一个数为 0, 如果有 0, 两者异或, 看另一个数是不是也是 0, 如果结果为 0, 说明另一个不为 0, 返回 0, 否则说明两个都是 0, 返回 1。如果都不是 0 且符号位相同返回 1。
4. logTwo: 本质上是求最高位 1 的位置, 用了二分法。检查高 16 位是否有 1, 如果有, 则说明最高位至少在第 16 位及以上, 偏移量加 16, 将数值右移 16 位, 没有 1 就不偏移。然后在剩下的低位中, 依次检查高 8、4、2、1 位, 每次检查结果转换为对应的偏移加到总结果中。最后用按位或把所有权重组合起来, 就是最高位 1 的索引。
5. byteSwap: 计算位移量, 字节索引乘 8 得到对应的比特偏移量。提取字节, 分别将目标字节右移到底部, 与上 0xFF 提取出来, 再左移到对方位置。构造掩码把原数值中原来的两个字节清零。最后把清零后的数值与提取出的字节进行按位或来拼接。
6. reverse: 循环分别算出低位和对应的高位索引, 提取对应位置的位, 移位到对方位置。构造掩码清空原数值在这两位的原值, 最后按位或上交换后的位。
7. logicalShift: 算术右移制造出高位有 n+1 个 1, 再左移一位挤掉最左边的一个 1 并在右侧补 0, 此时高位正好有 n 个 1, 取反得到高位全 0 其余全 1 的掩码, 与算术右移后的原数值相与。
8. leftBitCount: 求左侧连续 1 等价于取反后求最高位 1 的位置, 取反后利用二分法寻找最高位 1 所在的位置。
9. float_i2f: 提取并保存符号位, 负数转正数。循环左移直到最高位为 1 并记录左移次数。剩下的高 23 位作尾数, 最低 8 位是被挤出的舍弃部分。如果舍弃部分大于 128 尾数进位, 等于 128 则看尾数最低位, 为 1 进位为 0 不进, 最后根据左移次数算出阶码, 拼接符号位、阶码和尾数。
10. floatScale2: 提取原符号位和阶码, 阶码加 1。阶码为 256 即无穷或非数字受影响直接返回。阶码为 1 即非规格化数, 直接连同尾数一起左移 1 位, 加上符号位返回。阶码为 255 即溢出为无穷大。正常规格化数清空原阶码部分, 按位或上新阶码即可。
11. float64_f2i: 提取符号和阶码。阶码大于等于 1054 说明越界返回溢出值, 小于 1023 说明绝对值小于 1 返回 0。将跨越两个变量的尾数拼接, 并补上隐藏的最高位放置在第 30 位。根据实际指数将拼接后的尾数右移到正确位置, 丢弃小数。负数则利用取反加一求补码处理符号。
12. floatPower2: 浮点数阶码部分决定了 2 的幂次, 真实指数为 x, 阶码为 127 加 x。x 大于等于 128 阶码达到 255 发生上溢, 返回正无穷。x 小于等于负 150 超出非规格化数最小范围发生下溢, 返回 0。非规格化数范围内靠尾数部分的右移来表示极小数。规格化数直接将算出的阶码左移 23 位, 尾数全 0 即可。